

Engenharia de Software – PUC Campinas

Algoritmos e Linguagens de Programação

Prof. Dr. Paulo Cesar de Alvarenga Lucci

Structs

Exercícios

○ ANTES DE COMEÇAR OS EXERCÍCIOS:

- Neste momento do curso, o foco é o desenvolvimento de funções — trechos de código bem definidos, reutilizáveis e independentes.
- Cada exercício desta lista descreve um problema que deve ser resolvido com uma função bem projetada.
- Para isso, siga estas orientações:
 - Comece escrevendo a função. Não comece pelo main. O main serve apenas para testar a função — e não o contrário.
 - Pense na função como uma "caixa-preta": ela deve receber somente os parâmetros estritamente necessários para executar sua tarefa e retornar um resultado (se for o caso), sem depender de variáveis externas ou interações com o usuário.
 - A função não deve imprimir nada (a menos que o enunciado peça). O teste da função com printf, scanf ou main() é feito à parte.
 - Seja direto. A função deve ser concisa, clara e focada. Evite colocar dentro dela partes do código que pertencem à lógica do programa principal.
 - Use o main() apenas como espaço de testes: você pode escrever diversos printf ali para verificar se a função está funcionando como deveria.
- Ao pensar cuidadosamente sobre os parâmetros e o retorno da função, você estará praticando um dos

Exercícios

- 1) Crie uma `struct Pessoa` com nome e idade. Faça uma função que receba uma `Pessoa` por valor e imprima seus dados.
- 2) Altere o exercício anterior para que a função receba a `Pessoa` por referência imprima seus dados.
- 3) Crie uma função que receba duas `Pessoa` e retorne o nome da mais velha.
- 4) Escreva uma função que crie e retorne uma `Pessoa` inicializada diretamente (sem `strcpy`).
- 5) Faça uma função que receba uma `Pessoa` e retorne `true` se a idade for maior ou igual a 18, e `false` caso contrário.
- 6) Crie uma função que receba um vetor de `Pessoa` e mostre todos os nomes cadastrados.
- 7) Faça uma função que receba um vetor de `Pessoa` e um nome, e retorne o índice da pessoa (ou -1 se não encontrar).
- 8) Crie uma função que receba um vetor de `Pessoa` e retorne a mais velha.

Exercícios

- 9) Crie uma função que receba um vetor de `Pessoa` e reordene esse vetor em ordem alfabética de nomes (utilize a função `int strcmp(string str1, string str2)` ensinada em strings).
- 10) Crie uma `struct Aluno` com nome e três notas (float). Faça três funções:
 - Ler dados do aluno.
 - Calcular média aritmética.
 - Verificar se está aprovado (média ≥ 6).
- 11) Crie uma `struct Turma` com um vetor de até 30 `Aluno`. Escreva uma função que retorne um vetor com o(s) melhor(es) aluno(s) da turma. Obs.: O melhor aluno é aquele de maior média. Se a maior média for compartilhada por `N` alunos, o vetor terá tamanho `N`. Se for um único aluno que apresenta a maior média, o vetor terá tamanho 1.

Exercícios

- 12) Crie uma `struct Produto` (código, nome, preço, quantidade) e `struct Loja` que guarda em si um vetor de `Produto`. Faça três funções:
 - Cadastrar vários `Produto` em uma `Loja`.
 - Buscar `Produto` pelo código na `Loja`
 - Atualizar quantidade de um `Produto` buscado pelo código.